MSFS Live Traffic Utility

Technische Projektdokumentation

Desktopanwendung · ADS-B-Livekarte · MSFS/SimConnect · PHP-Proxy · C++-Collector

Dokumentstand: 11. Oktober 2026

Zielgruppe: Anwender, Entwickler und Proxy-Betreiber

Diese Unterlage beschreibt Bedienung, Datenfluss, Modularchitektur, Netzwerk-Failover, Einstellungen, Tests, Release-Build und den vollständigen Self-Hosting-Betrieb.

Quelle: Projekt-Repository PhW3130/MSFS-Live-Traffic-Utility-PUBLIC-BETA

Inhaltsverzeichnis

Inhaltsverzeichnis **2**

1. Projektüberblick **4**

1.1 Zweck 4

1.2 Dokumentierter Umfang 4

1.3 Betriebsarten 4

2. Systemarchitektur und Datenfluss **4**

2.1 Hauptkomponenten 4

2.2 Standardpfad und Ausweichstufen 5

2.3 Demand-gesteuerte Sammlung 5

3. Desktopoberfläche und Bedienung **5**

3.1 Karte, Tabelle und Suche 5

3.2 Darstellung und Export 6

3.3 Einstellungen und Speicherung 6

3.4 Startbildschirm und Diagnose 6

4. Microsoft-Flight-Simulator-Integration **7**

4.1 SimConnect und KI-Verkehr 7

4.2 Flugzeug- und Modellzuordnung 7

4.3 TCAS und Parkflugzeuge 7

5. Konfiguration und Umgebungsvariablen **7**

5.1 Proxy-URL und Token 7

5.2 Wichtige Werte 8

6. ADS-B-Datenquellen und fachliche Regeln **8**

6.1 Abfragereihenfolge 8

6.2 adsb.lol 8

6.3 adsb.fi Open Data 9

6.4 OpenSky Network 9

6.5 Routen- und Kartendaten 9

7. PHP-Proxy und C++-Collector **9**

7.1 Rollen und Datenhaltung . . . 9
7.2 PHP-Endpunktreferenz . . . 10
7.3 Statusseite und Admin-Konsole . . . 11
8. Eigenen Proxy bauen und betreiben . . . 11
8.1 Voraussetzungen und Paketaufbau . . . 11
8.2 Eigene URL eintragen . . . 11
8.3 Privates Token erzeugen und erneuern . . . 12
8.4 C++-Collector bauen . . . 12
8.5 Collector-INI und CLI . . . 12
8.6 OpenSky-Credentials und Kontingent-Ledger . . . 13
8.7 Watchdog und Windows-Autostart . . . 13
8.8 PHP-Dateien bereitstellen . . . 13
9. Peer-Mesh: Wahl, Web-Relay und Grenzen . . . 13
10. Repository und Modulübersicht . . . 14
10.1 Qt- und JavaScript-Kommunikation . . . 14
11. Entwicklung, Tests und Release . . . 15
11.1 Python-Abhängigkeiten . . . 15
11.2 Headless-Python-Tests . . . 15
11.3 C++- und PHP-Tests . . . 15
11.4 Windows-Pakete und Releases . . . 15
11.5 Mitwirken . . . 16
12. Sicherheit, Datenschutz und Betrieb . . . 16
13. Fehlerbehebung . . . 16
14. Roadmap . . . 17
15. Drittanbieter, Lizenz und Haftungsausschluss . . . 17
16. Begriffe . . . 18
Anhang A. Lizenzhinweis für eingebettete PDF-Schriften . . . 19

1. Projektüberblick

1.1 Zweck

MSFS Live Traffic Utility ist eine Python-Desktopanwendung zur Anzeige öffentlich verfügbarer ADS-B-Flugbewegungen auf einer interaktiven Karte. Optional kann sie passende, lokal installierte Flugzeugmodelle über SimConnect als KI-Verkehr in Microsoft Flight Simulator injizieren. Karte und Tabellenansicht bleiben auch ohne laufenden Simulator nutzbar.

Die Desktopoberfläche ist eine Qt-Anwendung auf Basis von PySide6. Die Kartenoberfläche läuft in QtWebEngine und verwendet Leaflet; die Kommunikation zwischen Python und JavaScript erfolgt über QtWebChannel. Tkinter ist nicht die GUI-Technologie dieses Projekts.

1.2 Dokumentierter Umfang

Die aktuell dokumentierte aktive Regionalquellen-Kette umfasst ausschließlich **adsb.lol → adsb.fi Open Data → OpenSky Network**. Direkte Abfragen dieser Quellen sind die letzte Ausweichstufe hinter dem zentralen Proxy und dem Web-Relay-Peer-Mesh.

Die Anwendung und insbesondere KI-Injektion sowie Modellzuordnung bleiben experimentell. Ergebnisse der öffentlichen Datenquellen, Flugzeugzuordnung und MSFS-Spawns sind nicht garantiert.

1.3 Betriebsarten

Betriebsart	Voraussetzungen	Ergebnis
Windows-Release	Windows 10/11; Internetzugang	Entpacken und `MSFS Live Traffic Utility.exe` starten.
Quellcode, Kartenmodus	Python 3.10+, `requirements.txt`, Internet	Qt-Karte und ADS-B-Verkehr; Simulatorverbindung optional.
Quellcode, MSFS-Integration	Zusätzlich MSFS 2020 oder 2024, nutzbare SimConnect-DLL/Verbindung und kompatible lokale Flugzeugmodelle	Spielerposition, TCAS und optionale KI-Verkehrsinjektion.
Eigener Proxy	Windows-Collector, PHP-Webhost, gemeinsames privates Token	Ein Collector bedient die aktiven Kartenbereiche mehrerer Clients.

2. Systemarchitektur und Datenfluss



2.1 Hauptkomponenten

1. **Qt-Client:** `main.py` stellt Fenster, Werkzeugleisten, Flugzeugtabelle, Detailbereich und QWebEngine-Karte bereit. `settings_dialog.py` enthält den mehrseitigen Einstellungsdialog.

2. **Traffic-Engine:** `traffic_core.py` fragt Verkehr ab, normalisiert Antworten, verwaltet Filter, Suche, Auswahl, Flugspuren und Zustands-Snapshots. Die Engine ist unabhängig von der Qt-Oberfläche testbar.
3. **Web-Relay:** `proxy/index.php` ist PHP-Endpunkt, öffentlicher Status und Admin-Konsole. Es speichert regionale Antworten, Demand, Client-Heartbeats und Mesh-Zustand als Dateien; eine Datenbank ist nicht erforderlich.
4. **Collector:** `proxy/ltu_collector.cpp` ist ein Windows-Programm auf dem Betreiber-PC. Es fragt aktive Demand-Regionen beim PHP-Relay ab, ruft passende Upstream-Daten ab und sendet Ergebnisse token-geschützt zurück.
5. **Peer-Mesh:** `peer_net.py` führt Heartbeats aus und setzt die vom Server bestimmte Rolle um. Ein gewählter Client kann im Fehlerfall Verkehr über die Webseite an die übrigen Clients weiterreichen.
6. **MSFS-Anbindung:** `msfs_ai.py` verwaltet SimConnect, Spielerposition, AI-Spawns und Parkflugzeug-Füller in einem von der GUI unabhängigen Worker.

2.2 Standardpfad und Ausweichstufen

Fertig konfigurierte Clients sind auf den zentralen Community-Proxy ausgelegt. Dieser bündelt Upstream-Abfragen und stellt unter anderem öffentlichen Status und gemeinsam gespeicherte, einstündige Flugspuren bereit. Die öffentliche Statusseite benötigt kein Token.

Die Ausweichreihenfolge ist:

1. **Community- oder eigener PHP-Proxy.** Der Client fragt den sichtbaren Kartenausschnitt ab.
2. **Web-Relay des Peer-Meshes.** Wenn Collector-Daten veraltet sind oder mindestens die Hälfte der frischen Client-Heartbeats einen nicht verfügbaren bzw. mit HTTP 429 antwortenden Proxy meldet, bestimmt der Webserver einen geeigneten Relay-Client. Der Relay-Verkehr läuft weiterhin über die PHP-Webseite; Clients bauen keine direkte Verbindung zueinander auf.
3. **Direkte Quellen auf dem Client:** adsb.lol, danach adsb.fi Open Data, danach anonyme OpenSky-Abfrage. Diese Stufe wird erst genutzt, wenn Proxy und Web-Relay nicht verfügbar sind.
4. **Letzter lokaler Cache.** Wenn keine frische Quelle Daten liefert, kann der laufende Prozess seinen zuletzt empfangenen Verkehr wiederverwenden.

Das Mesh benötigt keine eingehenden Ports oder Router-Portweiterleitung. Ist der PHP-Webhost selbst nicht erreichbar, kann auch die Mesh-Wahl nicht laufen; die Clients greifen dann auf ihre direkten Quellen zurück.

2.3 Demand-gesteuerte Sammlung

Ein Client meldet über `action=traffic` nur den sichtbaren Kartenausschnitt oder einen angefragten Kreis. PHP bildet daraus eine kurzlebige, grob gerasterte Demand-Region. Der Collector fragt `action=demand` standardmäßig alle fünf Sekunden ab und sammelt nur für aktive Bereiche. Eine Demand-Region verfällt nach 20 Sekunden ohne neue Verkehrsanfrage; wenn keine Regionen und keine explizite globale Suche aktiv sind, pausiert der Collector sämtliche Upstream-Abfragen.

Demand wird in 0,25-Grad-Zellen zusammengefasst. Die private Demand-Datei liegt im System-Temp-Verzeichnis des PHP-Prozesses, außerhalb des Webroots, wird dateigesperrt und mit Modus `0600` angelegt. Öffentliche Statusantworten enthalten weder exakte Demand-Geometrie noch rohe Client-IP-Adressen. Es können höchstens 64 aktive Regionen verwaltet werden.

3. Desktopoberfläche und Bedienung

3.1 Karte, Tabelle und Suche

Die Hauptansicht enthält Leaflet-Karte, Live-Tabelle und Detailbereich. Die Karte startet standardmäßig in der Gesamtansicht. Die Suche kann Rufzeichen, ICAO-Hex, Flugzeugtyp oder Registrierung verwenden; Airline- und Typfilter ergänzen sie. Weitere Filter umfassen Mindesthöhe, Bodenverkehr, Beschriftungen, Folgen und Spuren.

- **Start:** eingegebene Flugnummer verfolgen.

- **All aircraft:** zur Gesamtansicht zurückkehren.
- **Einfachklick** auf Kartensymbol oder Tabellenzeile: Flugzeug auswählen.
- **Doppelklick** auf Tabellenzeile: Flugzeug folgen und heranzoomen.
- **Esc:** Auswahl aufheben.
- **+ / -:** Kartenzoom ändern.
- **Rechtsklick auf die Karte:** nächstgelegenes Flugzeug auswählen.
- **Enter in der Suche:** ersten Treffer auswählen und anspringen.

Die Tabelle wird parallel zur Karte aktualisiert. Ausgewählte Flugzeuge zeigen verfügbare Airline-, Kategorie-, Routen- und Navigationsdaten. Die Routensuche fragt adsbdb ab, kann eine Großkreisroute zeichnen und Start-/Zielflugplatzmarkierungen ergänzen.

3.2 Darstellung und Export

Flugzeugsymbole sind gedrehte SVG-Silhouetten für Verkehrsflugzeuge und Hubschrauber. Farben kodieren die Höhe; Bodenverkehr wird abgedunkelt. Squawk 7500, 7600 oder 7700 sowie ein ADS-B-Notfallstatus können markiert werden. Notfall-Popups und -Töne beziehen sich auf sichtbaren Verkehr und vermeiden doppelte Alarme.

Für die Basiskarte stehen OpenStreetMap, dunkle CARTO-Karten, OpenTopoMap und ein eigener Tile-Server zur Auswahl. Eigene Tile-URLs müssen `{z}`, `{x}` und `{y}` enthalten; korrekte Provider-Attribution und Nutzungsbedingungen sind einzuhalten. Die aktuelle Verkehrsliste kann als semikolongetrennte, Excel-freundliche CSV exportiert werden.

Flugspuren werden inkrementell und längenbegrenzt angezeigt. Der Proxy speichert aus Collector-/Relay-Pushes und frischen Regionalantworten Positionsstichproben für bis zu eine Stunde. Ein Track kann über ICAO-Hex oder Rufzeichen abgerufen und beim Auswählen asynchron nachgeladen werden. Die PHP-Implementierung tastet typischerweise höchstens alle zehn Sekunden ab und begrenzt die Länge auf 360 Punkte pro Flugzeug.

3.3 Einstellungen und Speicherung

Der PySide6-Einstellungsdialog ist mehrseitig. Er bearbeitet unter anderem Verbindung, Proxy und Mesh, Kartenstil und eigene Tiles, Kartenmittelpunkt und Zoom, Theme- und Höhenfarben, Icon-Größen, Filter und Aktualisierungsintervall sowie Simulatoroptionen. Die Werte und Fenstergeometrie werden benutzerspezifisch über QSettings gespeichert; Änderungen werden, soweit möglich, sofort übernommen. Ein Wechsel des Community-Ordners oder der SimConnect-DLL erfordert einen Neustart.

Der Dialog kann vorherige Standardwerte anzeigen und wiederherstellen. Dabei bleiben gespeicherte Proxy-Zugangsdaten, Mesh-Name und lokale MSFS-Pfade erhalten. `colors.json` ist die Konfiguration für ausgelieferte Entwickler-/Distributoren-Standardwerte: Theme-, Label-, Notfall- und Akzentfarben, Höhenfarbverlauf, Icons, Karte, Proxy und Peer-Mesh.

3.4 Startbildschirm und Diagnose

Der animierte Startbildschirm zeigt tatsächliche Ladeschritte, darunter den Scan von `aircraft.cfg`-Dateien. Pro Programmstart entstehen zeitgestempelte Diagnosen in `logs/`. Sie erfassen unter anderem UI-Aktionen, Einstellungsänderungen, Quellenanfragen und Statuswechsel sowie MSFS- und Mesh-Ereignisse. Ist das Anwendungsverzeichnis nicht beschreibbar, verwendet die Anwendung ein Benutzer-Datenverzeichnis.

Die UI-Sprache wird anhand der Betriebssystem-Locale automatisch auf Deutsch oder Englisch gesetzt. Desktop-Statusanzeigen decken Proxy, adsb.lol, adsb.fi, OpenSky, Mesh-Rolle und MSFS ab: Grün bedeutet gesund, Orange HTTP 429, Rot ausgefallen und Grau inaktiv. Der aktuelle öffentliche Servicestatus wird ebenfalls auf der Webkarte angezeigt. Die Traffic-Engine bleibt unabhängig von Qt testbar.

4. Microsoft-Flight-Simulator-Integration

4.1 SimConnect und KI-Verkehr

SimConnect ist für den Kartenmodus nicht notwendig. Bei erkannter Simulatorverbindung kann die Anwendung Spielerposition und Flugzeugstatus verwenden. KI-Injektion ist standardmäßig beim erkannten Simulator vorgesehen und kann über die Werkzeugleiste unabhängig von der Verkehrskarte ein- und ausgeschaltet werden. Die Verbindung wird erneut geprüft, falls der Simulator erst nach dem Programmstart läuft.

SimConnect benötigt eine verwendbare `SimConnect.dll`. Windows-Builds bündeln die Projektdatei; `MSFS_SIMCONNECT_DLL` kann auf eine lokale kompatible DLL verweisen. KI-Injektion verlangt außerdem MSFS 2020 oder 2024 und kompatible, lokal installierte Modelle. FSLTL-Basismodelle werden empfohlen, aber nicht mitgeliefert.

4.2 Flugzeug- und Modellzuordnung

`aircraft_config.py` sucht MSFS-Installationen anhand von `UserCfg.opt`-Dateien. Bei mehreren Installationen kann der Benutzer den gewünschten Community-Ordner wählen; ohne automatische Erkennung kann ein Community- oder übergeordneter Packages-Ordner manuell angegeben werden. `MSFS_COMMUNITY_PATH` und `MSFS_PACKAGES_PATH` überschreiben die automatische Erkennung.

Die Titel der installierten Flugzeuge werden aus `aircraft.cfg` indiziert. `model_map.py` liest die FSLTL-ModelMatchRuleSet-Regeln aus `modelmap.txt` und wählt passende Modelltitel aus. Die Titelzuordnung basiert auf älteren `aircraft.cfg`-Einträgen. SimConnect EX1 erhält einen vollständigen Titel und ein leeres Livery-Feld; dadurch wird das Standard-Livery-Modell verwendet. Eine deterministische Auswahl modularer MSFS-2024-Liveries ist nicht implementiert.

4.3 TCAS und Parkflugzeuge

Im TCAS-Modus wird die eigene Spielerposition auf der Karte dargestellt und die Ansicht auf den Spieler zentriert. Für das ausgewählte Verkehrsziel zeigt die Anwendung relative Peilung bzw. Uhrposition, Entfernung, Höhendifferenz sowie Steig-/Sinktrend.

Parkflugzeug-Füller sind künstliche, statische SimObjects am Boden und keine echten ADS-B-Flüge. Die Flugplatzerkennung hat folgende Priorität: `MSFS_AIRPORT_ICAO`, aktuelle SimConnect-Flugplatz-SimVars, danach eine Ableitung aus dem Live-Verkehr. Standardmäßig sind höchstens 50 Füller vorgesehen; als Standardmodell wird ein installiertes A20N-Modell gewählt. `MSFS_MAX_PARKED_FILLERS` setzt die Obergrenze und `MSFS_PARKED_FILLER_MODEL` überschreibt das Modell.

SimConnect stellt keine verlässliche Zahl freier Parkpositionen bereit. Die Füller-Erzeugung endet beim ersten abgelehnten Spawn oder an der konfigurierten Sicherheitsgrenze. Das Verhalten ist bestmöglich, keine Zusage für das Auffüllen jeder Position. Ein geparkter Live-Flug mit bekannter Route wechselt in den Abflugmodus, wenn die gemeldete Geschwindigkeit mindestens 25 Knoten erreicht oder die Steigrate über 200 Fuß/min liegt; er wird nach standardmäßig 900 Sekunden Wartezeit ohne Abflug wieder entfernt. Für die routegestützte Erkennung eines Flugplatzes in der Nähe des Spielers gilt standardmäßig ein Radius von 8 km. Aktive Füllerparameter sind in `msfs_ai.py` und `routes.py` konfigurierbar.

5. Konfiguration und Umgebungsvariablen

5.1 Proxy-URL und Token

Die URL-Auflösung für den Desktop erfolgt in dieser Reihenfolge:

1. `LTU_PROXY_URL` (Umgebungsvariable),
2. gespeicherte QSettings-URL,
3. erste Nicht-Kommentarzeile in `page.txt`,
4. `colors.json` unter `proxy.url`.

`page.txt` ist gitignored und kann neben dem Quellskript oder der gepackten EXE liegen. `page.example.txt` ist eine Vorlage. `LTU_PROXY_TOKEN` überschreibt das gespeicherte Token; ein Token kann außerdem in den Einstellungen bzw. Standardwerten stehen. Öffentliche Traffic-Abfragen und die öffentliche Statusseite benötigen im Normalfall kein Collector-Token; geschützte Collector-Aktionen und die Admin-Konsole schon.

5.2 Wichtige Werte

Variable / Schlüssel	Bedeutung
`LTU_PROXY_URL`	Proxy-Endpunkt mit höchster URL-Priorität.
`LTU_PROXY_TOKEN`	Überschreibt gespeichertes Proxy-Token.
`LTU_MESH=off`	Deaktiviert das Peer-Mesh.
`peer_net.enabled`, `peer_net.name`	Mesh-Aktivierung und optionaler Anzeigename in `colors.json` bzw. QSettings.
`MSFS_COMMUNITY_PATH`	Überschreibt die erkannte Community-Ordnerangabe.
`MSFS_PACKAGES_PATH`	Gibt einen Packages-Ordner an bzw. überschreibt den Standard.
`MSFS_SIMCONNECT_DLL`	Pfad zu einer lokalen SimConnect-DLL.
`MSFS_AIRPORT_ICAO`	Überschreibt die Erkennung des Parkflugzeug-Flugplatzes.
`MSFS_MAX_PARKED_FILLERS`	Obergrenze künstlicher Parkflugzeuge (Standard: 50).
`MSFS_PARKED_FILLER_MODEL`	Überschreibt das installierte A20N-Standardmodell.
`MSFS_SPAWN_RADIUS_KM`	Maximale Entfernung für Live-AI-Spawns vom Spieler (Standard: 35 km).
`MSFS_MAX_AI_AIRCRAFT`	Maximale Anzahl gleichzeitiger Live-AI-Flugzeuge (Standard: 25).
`MSFS_SPAWN_SETTLE_SEC`	Pause nach einem Spawn-Aufruf (Standard: 0,35 s).
`MSFS_PARKED_DEPART_GS`	Geschwindigkeitsschwelle zum Erkennen eines Abflugs aus Parkposition (Standard: 25 kt).
`MSFS_MAX_PARKED_WAIT_SEC`	Maximale Wartezeit eines Park-Füllers (Standard: 900 s).
`MSFS_AIRPORT_RADIUS_KM`	Nähe-Radius der routengestützten Flugplatzerkennung (Standard: 8 km).

`colors.json` kann Tile-URL, Attribution und maximale Zoomstufe festlegen. Eine eigene Tile-URL muss `{z}`, `{x}`, `{y}` enthalten. Die Nutzung des Tile-Servers und dessen Attribution sind unabhängig von ADS-B-Datenbedingungen.

6. ADS-B-Datenquellen und fachliche Regeln

6.1 Abfragereihenfolge

Normalbetrieb: Proxy-Collector; bei Proxy-Ausfall oder HTTP 429 folgt das Web-Relay-Peer-Mesh. Direkte Client-Abfragen sind die letzte Ausweichstufe und erfolgen in der Reihenfolge adsb.lol, danach adsb.fi Open Data, danach anonyme OpenSky-Abfrage. Bei HTTP 429 wird die betroffene Clientquelle vorübergehend übersprungen; der Standard-Cooldown beträgt 60 Sekunden, ein vom Anbieter gemeldetes `Retry-After` wird bis maximal 24 Stunden berücksichtigt. Liefert keine Stufe frische Daten, kann der laufende Client-Prozess den letzten Cache verwenden. Ein expliziter weltweiter Suchauftrag wird nicht durch einen dauerhaften Global-Feed erfüllt; er stellt eine einmalige OpenSky-Suche über den Proxy-Collector in die Warteschlange.

6.2 adsb.lol

adsb.lol wird für regionale Abfragen über die v2-Route `/v2/lat/{lat}/lon/{lon}/dist/{dist}` verwendet. Der Betreiber beschreibt dynamische, lastabhängige Begrenzungen statt eines festen Tageskontingents. 4xx-Antworten können auf ungültige Anfragen hinweisen; eine Schlüsselpflicht kann sich ändern. Maßgeblich ist die offizielle API-Dokumentation.

6.3 adsb.fi Open Data

Regionaler Fallback ist die öffentliche v3-Route `/api/v3/lat/{lat}/lon/{lon}/dist/{dist}`. Die Distanz wird in nautischen Meilen angegeben und im öffentlichen Proxybereich auf 250 NM begrenzt. Öffentliche Abfragen sind auf eine Anfrage pro Sekunde begrenzt. Der Dienst ist für persönliche, nicht-kommerzielle Nutzung vorgesehen; die Betreiber erwarten Quellenangabe. Siehe Open-Data-Dokumentation.

6.4 OpenSky Network

Der Collector kann OpenSky OAuth2 `client_credentials` für regionale `/api/states/all`-Abfragen verwenden. Zugangsdaten liegen ausschließlich in einer lokalen JSON-Datei neben dem Collector; der Client erhält sie nicht. Die anonyme OpenSky-Direktabfrage des Desktopclients ist getrennt von diesen Collector-Zugangsdaten.

- Regionale und explizite globale Collector-Abfragen teilen ein persistentes `opensky_quota.txt`-Ledger neben der EXE.
- Das Ledger reserviert jeden Versuch vor dem Netzwerkaufruf und bleibt über Neustarts erhalten. Es darf nicht gelöscht werden, wenn das Tageslimit dauerhaft eingehalten werden soll.
- Der Mindestabstand beträgt 120 Sekunden; maximal sind 700 Versuche im rollierenden 24-Stunden-Fenster vorgesehen. Das Ledger arbeitet bei Speicherfehlern geschlossen (keine weiteren Abfragen).
- HTTP-429-Wartezeiten berücksichtigen `X-Rate-Limit-Retry-After-Seconds` und `Retry-After` als Sekundenwert oder HTTP-Datum, maximal einen Tag.
- Laut OpenSky kostet `/states/all` je nach BBox-Fläche 1–4 Credits. Dokumentierte Kontingente sind 400 Credits/Tag anonym pro IP, 4.000 für Standardkonten und 8.000 für aktive Feeder. Diese Anbieterwerte können sich ändern; aktuelle OpenSky-REST-Dokumentation beachten.
- Der direkte anonyme Desktop-Fallback begrenzt den Radius auf 250 km und Abfragen auf höchstens eine je 15 Minuten pro Prozess. Das sind maximal 96 Abfragen bzw. 384 Credits in 24 Stunden bei vier Credits je Abfrage. Der Timer wird beim Neustart zurückgesetzt und koordiniert keine weiteren Nutzer hinter derselben öffentlichen IP.

OpenSky-State-Arrays werden anhand dokumentierter Indizes normalisiert: `baro_altitude` (Index 7) und `geo_altitude` (13) bleiben getrennt und werden von Metern in Fuß umgerechnet; `velocity` (9) wird von m/s in Knoten, `vertical_rate` (11) von m/s in Fuß/min umgerechnet; `on_ground` ist Index 8. Die Live-API ist für Forschung und nicht-kommerzielle Nutzung vorgesehen.

6.5 Routen- und Kartendaten

Routeninformationen kommen über adsbdb (`routes.py`) und sind kein weiterer ADS-B-Live-Feed. Andere kostenpflichtige Luftfahrtdaten-Feeds wie ADS-B Exchange oder FlightAware sind nicht integriert. Kartentiles sind ebenfalls ein unabhängiger Dienst: OpenStreetMap, CARTO, OpenTopoMap und selbst konfigurierte Anbieter haben eigene Verfügbarkeit, Nutzungsbedingungen und Attribution.

7. PHP-Proxy und C++-Collector

7.1 Rollen und Datenhaltung

Der PHP-Endpunkt `proxy/index.php` nimmt öffentliche Client-Anfragen an, protokolliert grobe aktive Nachfrage, speichert Collector-Antworten und stellt Status-/Adminseiten bereit. `proxy/client_demand.php` verwaltet kurzlebige Demand-Regionen; `proxy/aircraft_tracks.php` speichert sampled Flugspuren. Der C++-Collector nutzt Windows WinHTTP und benötigt keine zusätzlichen C++-Bibliotheken. Er verarbeitet aktive Client-Nachfrage, ruft Regionalquellen ab und schiebt Ergebnisse mit `X-LTU-Token` an PHP.

PHP speichert `ltu_index.json`, `ltu_source_status.json`, `ltu_clients.json`, `ltu_mesh.json`, `ltu_relay.json`, komprimierte `ltu_r_<region>.json.gz`-Regionaldaten und `ltu_h_<region>_<zeitstempel>.json.gz`-Replay-Snapshots. Der Push-Log ist im Index enthalten. Das einstündige Track-Verzeichnis und Demand liegen im System-Temp-Verzeichnis außerhalb des Webroots. Es ist keine Datenbank erforderlich. PHP muss in seinem Laufzeitverzeichnis und im System-Temp-Verzeichnis schreiben dürfen; direkte HTTP-Downloads interner JSON-/GZip-Dateien müssen serverseitig blockiert werden.

Regionale Demand-Einträge verfallen nach 20 Sekunden, die öffentliche Demand-Obergrenze beträgt 64 Regionen und PHP schreibt denselben Demand-Bereich höchstens einmal je fünf Sekunden neu. Der Collector pollt standardmäßig alle 5 Sekunden; eingestellte Intervalle werden im Demand-Poll auf 2–10 Sekunden begrenzt. Regionale Provider werden feedweit mindestens eine Sekunde auseinander gehalten. Das maximale öffentliche Regionalgebiet beträgt 463 km (250 NM). PHP behandelt Regionalsnapshots standardmäßig nach 30 Sekunden als veraltet (`LTU_MAX_AGE`); Push-Metadaten können quellenspezifische Frischefenster setzen. Quellstatus gilt ohne Override 90 Sekunden als frisch (`LTU_SOURCE_FRESH`). Optionale `region=LAT,LON,RADIUS_KM`-Einträge in der Collector-INI oder CLI begrenzen nur aktuelle Client-Nachfrage und starten keine eigenen Upstream-Abfragen.

7.2 PHP-Endpunktreferenz

Endpunkt	Zugriff	Zweck
`GET ?action=traffic&bbox=N,S,E,W`	Client, öffentlich	Liefert nur Regionen im sichtbaren Kartenausschnitt; meldet `demand_recorded`. Kreisform `lat`, `lon`, `dist` bleibt kompatibel.
`POST ?action=push&region=<id>`	Collector, `X-LTU-Token`	Nimmt ADS-B-`ac[]` oder OpenSky-Daten an; Payload ist auf 30.000 Flugzeuge begrenzt.
`POST ?action=source_status&region=<id>`	Collector, `X-LTU-Token`	Meldet Abruffehler und begrenzte Metadaten wie HTTP-Status, Modus, Quelle und Frischefenster.
`GET ?action=demand`	Collector, `X-LTU-Token`	Gibt aktive, grob gerasterte Demand-Regionen mit opaken IDs zurück.
`GET ?action=global_search`	Client, öffentlich; Collector verarbeitet Queue	Stellt nur nach expliziter Suche eine globale OpenSky-Abfrage an; zentral höchstens alle 120 Sekunden, frischen Snapshot bis zu 180 Sekunden wiederverwenden.
`POST ?action=heartbeat` / `POST ?action=logout`	Client, kein Token	Registriert Status/Mesh-Heartbeat bzw. meldet den Client beim normalen Beenden ab.
`POST ?action=relay_push` / `GET ?action=relay_pull`	Gewähltes Relay / Clients	Veröffentlicht bzw. liefert vom Webserver gefilterte Relay-Daten. Es wird keine direkte Clientverbindung geöffnet.
`GET ?action=healthdata` / `?action=statuspage`	Öffentlich, kein Token	Bereinigte Statusdaten als JSON bzw. öffentliche HTML-Statusseite. Ohne `action` ist die Statusseite der Standard.
`GET ?action=status`	Öffentlich, kein Token	Regionsalter/-anzahl und bereinigter Push-Log.
`GET ?action=console`	Admin, gemeinsames Token	Geschützte Konsole mit Regionen, Verkehr, Push-Log, Karte, Push-Diagramm und Replay.
`GET ?action=hist&region=<id>&ts=<unix>`	Admin-Konsole, Token	Holt einen regionalen Replay-Snapshot.
`GET ?action=track&hex=<icao>` / `?action=track&flight=<callsign>`	Client, öffentlich	Liefert die gespeicherte einstündige Flugspur per ICAO-Hex oder Rufzeichen.

Wenn PHP Demand nicht speichern kann, liefert `action=traffic` trotzdem `demand_recorded:false`; der Qt-Client protokolliert den Fehler einmal, bis eine erfolgreiche Anfrage die Demand-Erfassung wiederherstellt. Das bedeutet, dass der Collector für diese Anfrage unter Umständen nicht mit frischem Demand versorgt wurde.

7.3 Statusseite und Admin-Konsole

Die öffentliche Statusseite ist ohne gemeinsames Token erreichbar. Sie lädt Statusdaten alle fünf Sekunden und zeigt Proxy-/Collector-Erreichbarkeit, Regionen und Flugzeuge, aktive Nachfrage, Quellstatus, Clients sowie gewähltes Relay und Relay-Frische. Eine Matrix Client × Komponente zeigt Proxy, adsb.lol, adsb.fi, OpenSky und MSFS. Grün bedeutet gesund, Orange Rate-Limit/HTTP 429, Rot ausgefallen und Grau noch nicht gemeldet. Quellstatus hat quellenspezifische Frischefenster und kann als veraltet markiert werden; ohne aktive Nachfrage wird pausierte Sammlung angezeigt.

Die Admin-Konsole liegt unter `?action=console` und verlangt das private gemeinsame Token. Sie zeigt Regionsstatus, aktuelle Flugzeuge und Push-Log, Live-Karte mit Regionengrenzen und höhengefärbtem Verkehr, Push-Zählerdiagramm sowie Replay-Steuerung je Region mit Zeitregler und Play-Taste. Die Oberfläche aktualisiert Daten mit `fetch` ohne vollständigen Seitenreload. Pro Region entsteht jede Minute ein reduzierter Positions-/Höhensnapshot; etwa 90 Minuten Replay-Material bleiben erhalten. Die eigenständige Track-API hält Flugzeugspuren eine Stunde vor.

Collector-Quellstatus beschreibt die Upstream-Abrufe, nicht die Erreichbarkeit des PHP-Hosts selbst. Ein veralteter Quellstatus beweist daher keinen Ausfall der Webseite.

8. Eigenen Proxy bauen und betreiben

8.1 Voraussetzungen und Paketaufbau

Für eine unabhängige Instanz werden benötigt:

- Windows 10/11 für Collector und gepackte Desktop-App.
- MinGW-w64 `g++` oder Visual Studio Build Tools mit C++-Workload.
- Python 3.10+ und Projektanforderungen für den Desktop-Build.
- PHP-Webhost (die CI-/Proxytests verwenden PHP 8.2), beschreibbares Proxy-Verzeichnis und beschreibbares System-Temp-Verzeichnis.
- OpenSky-OAuth2-Client-Credentials auf dem Collector-PC, wenn OpenSky als dritte Regionalquelle genutzt werden soll.
- Ein starkes, privates gemeinsames `LTU_TOKEN` für Collector↔PHP und Admin-Konsole.

`build_all.bat` im Projekt-Root löscht `rea/` und erzeugt es neu. Vor dem erneuten Build lokale Einstellungen oder Zugangsdaten daraus sichern. Die Ausgaben:

Pfad	Inhalt / Ziel
`rea/php/`	`index.php`, `client_demand.php`, `aircraft_tracks.php`; gemeinsam auf den Webhost kopieren.
`rea/proxy/`	`ltu_collector.exe`, `ltu_collector.ini`, OpenSky-Vorlage und Watchdog-/Task-Skripte; auf den Collector-PC kopieren.
`rea/user/MSFS Live Traffic Utility/`	Eigenständiger Qt-Client und `page.txt`; an Client-PCs verteilen.
`rea/LIESMICH.txt`	Vom Build erzeugte Paketnotizen. Der CI-Build ergänzt `rea/TOKEN-SETUP.txt`.

Die Verteilung enthält absichtlich keine privaten OpenSky-Zugangsdaten. Verteile echte Zugangsdaten weder an Clients noch in Webroot, Releases oder Git.

8.2 Eigene URL eintragen

Vor dem Self-Hosting-Build die URL in `LTU_URL` in `build_all.bat` und in `url=` in `proxy/ltu_collector.example.ini` auf den eigenen Host setzen. Beispiel – **durch die eigene Instanz ersetzen**:

```
http://your-server.example/index.php
```

Nach dem Build sowohl `rea/proxy/ltu_collector.ini` als auch `rea/user/MSFS Live Traffic Utility/page.txt` prüfen. Den Collector oder Client erst starten, wenn beide Dateien auf den eigenen Host zeigen. Für untrusted Netze HTTPS am Webhost konfigurieren: Das gemeinsame Token darf nicht ungeschützt über eine unsichere Klartextverbindung übertragen werden. Die `page.txt`-Vorlage erwartet die URL in der ersten Nicht-Kommentarzeile.

8.3 Privates Token erzeugen und erneuern

`token.txt` liegt neben `build_all.bat` und ist gitignored. Fehlt die Datei, erzeugt `build_all.bat` ein kryptografisch sicheres 256-Bit-Token. `generate_token.bat` erzeugt bzw. erneuert die Datei manuell und fragt vor dem Überschreiben nach. Gültig ist eine einzelne Zeile mit 8–256 Zeichen aus `A-Z`, `a-z`, `0-9`, Punkt, Unterstrich, Tilde und Bindestrich.

`build_all.bat` setzt denselben Wert in `rea/php/index.php` (`LTU_TOKEN`) und `rea/proxy/ltu_collector.ini`. Geschützt sind Collector-Pushes, Demand-Polls, Quellstatusmeldungen, Replay und Admin-Konsole. Das Client-Paket erhält das Collector-Token nicht. `token.txt` privat halten, nicht committen und nicht in den Webroot kopieren. Nach einer Erneuerung `build_all.bat` erneut ausführen und sowohl PHP als auch Collector-INI neu deployen; alte Dateien authentifizieren nicht mehr.

8.4 C++-Collector bauen

Der Collector `proxy/ltu_collector.cpp` verwendet Windows WinHTTP und keine externen C++-Bibliotheken. `proxy/build_collector.bat` wählt MinGW oder MSVC, sofern installiert. Gleichwertige Befehle aus dem Ordner `proxy/`:

```
:: MinGW-w64
g++ -O2 -static -pthread ltu_collector.cpp -o ltu_collector.exe -lwinhttp

:: MSVC — im x64 Native Tools Command Prompt
cl /O2 /EHsc /MT ltu_collector.cpp /Fe:ltu_collector.exe /link winhttp.lib
```

`build_all.bat` kann Collector und Client gemeinsam bauen. `LTU_BUILD_CXX=cl` bzw. `LTU_BUILD_CXX=g++` wählt den Compiler explizit. Ohne C++-Compiler können PHP und Client weiterhin gebaut werden; der Collector fehlt dann und das Skript meldet einen Fehler. MSVC muss aus dem x64-Native-Tools-Terminal aufgerufen werden.

8.5 Collector-INI und CLI

`ltu_collector.ini` liegt neben `ltu_collector.exe`. Die Konfigurationsreihenfolge lautet **eingebaute Defaults → INI → Kommandozeile**, wobei spätere Werte Vorrang haben. Ohne Argumente kann der Collector doppelt angeklickt oder im Terminal gestartet werden.

```
url=http://your-server.example/index.php
token=HIER_EIN_PRIVATES_TOKEN_EINTRAGEN
interval=5
opensky_interval=120
opensky_credentials=credentials.json
; Optionale Allowlist — startet selbst keine Abfragen
; region=50.03,8.57,250
; region=52.52,13.40,250
```

INI-Regeln: pro Zeile `key=value`; `;` und `#` am Zeilenanfang sind Kommentare; Inline-Kommentare werden nicht interpretiert; Abschnittsüberschriften werden ignoriert. Unterstützte Schlüssel sind `url`, `token`, `interval`, `opensky_interval`, `opensky_credentials` und wiederholbare `region`-Einträge. Demand-Poll-Intervall Standard 5 Sekunden; akzeptierter Mindestwert 1, effektives Polling jedoch auf 2–10 Sekunden begrenzt. `opensky_interval` hat einen harten Mindestwert von 120 Sekunden. Relative Credential-Pfade beziehen sich auf den Ordner der EXE. `region=LAT,LON,RADIUS_KM` beschränkt aktuelle Client-Demand; es entstehen keine eigenständigen Abfragen. Das Maximum beträgt 463 km/250 NM.

```
ltu_collector.exe --url http://your-server.example/index.php --token PRIVATES_TOKEN --interval 5
ltu_collector.exe --region 50.03,8.57,250 --region 52.52,13.40,250
ltu_collector.exe --lat 50.03 --lon 8.57 --radius 250
```

`--region` ist wiederholbar und ersetzt CLI-seitig alle INI-Regionen. `--lat`/`--lon` mit optionalem `--radius` sind eine Kompatibilitätsabkürzung für eine Region. Radien sind Kilometer und werden auf das öffentliche Maximum begrenzt. `--help` zeigt die Nutzung.

8.6 OpenSky-Credentials und Kontingent-Ledger

Für OpenSky nur auf dem Collector-PC `rea/proxy/credentials.example.json` bzw. `proxy/credentials.example.json` nach `credentials.json` neben die EXE kopieren und `clientId` sowie `clientSecret` eintragen:

```
{
  "clientId": "EIGENE_OPENSKY_CLIENT_ID",
  "clientSecret": "EIGENES_OPENSKY_CLIENT_SECRET"
}
```

Die OpenSky-Zugangsdaten sind vom `LTU_TOKEN` getrennt: OAuth authentifiziert gegenüber OpenSky; das private LTU-Token schützt Collector↔PHP. Der Collector fordert OAuth-Tokens an und erneuert sie; Zugangsdaten und Bearer-Tokens bleiben im Collector-Prozess, werden nicht an Clients gesendet und nicht protokolliert. Die Datei auf den Collector beschränken, außerhalb von Git/Releases/Webroot halten. Ein eigener Dateipfad kann mit `opensky_credentials=` gesetzt werden.

`opensky_quota.txt` liegt neben der EXE, gilt gemeinsam für regionale und explizite globale Collector-Abfragen und überlebt Neustarts. Es protokolliert Versuche vor dem Senden. Nicht löschen, wenn das Tagesbudget über Neustarts erhalten bleiben soll. Bei nicht lesbarem oder nicht beschreibbarem Ledger wird geschlossen abgebrochen. Der Collector hält mindestens 120 Sekunden zwischen Versuchen und maximal 700 Versuche pro rollierenden 24 Stunden ein; Details zur Credit-Abrechnung und aktuelle Provider-Quoten stehen im Abschnitt OpenSky.

8.7 Watchdog und Windows-Autostart

Manueller Watchdog-Start aus `rea/proxy` bzw. dem Ordner neben der EXE:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\run_collector_watchdog.ps1
```

Der Watchdog startet den Collector verborgen, startet nach Fehlern neu und führt standardmäßig nach 24 Stunden einen täglichen Neustart aus. Er schreibt `logs/watchdog.log` sowie pro Lauf `collector-*.stdout.log` und `collector-*.stderr.log`; die Run-Logs sind auf höchstens 100 aktuelle Dateien bzw. 14 Tage begrenzt. `watchdog.log` rotiert ab 1 MB zu einer Sicherung `watchdog.log.1`. Beim direkten Doppelklick ist die Ausgabe nur im Konsolenfenster.

Autostart des Watchdogs beim Anmelden des aktuellen Windows-Benutzers installieren bzw. entfernen:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\install_collector_task.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File .\remove_collector_task.ps1
```

Die Aufgabe läuft mit eingeschränkten Rechten. Beim Entfernen werden die zugehörige Aufgabe und der passende Collector-Prozess beendet; vorhandene Logs bleiben erhalten. Collector und Peer-Mesh nutzen ausgehendes HTTP zum PHP-Host; eingehende Ports sind nicht erforderlich.

8.8 PHP-Dateien bereitstellen

Für eine manuelle Bereitstellung gemeinsam hochladen:

```
proxy/index.php
proxy/client_demand.php
proxy/aircraft_tracks.php
```

Das `LTU_TOKEN` in `index.php` muss dem `token` in der Collector-INI entsprechen. `build_all.bat` trägt das Token automatisch ein. PHP benötigt Schreibzugriff auf den Relay-Ordner und den System-Temp-Ordner. Interne JSON-/GZip-Dateien und Snapshots dürfen nicht direkt über HTTP abrufbar sein; Zugriff ausschließlich über `index.php` erlauben. Öffentliche Statusantworten geben weder Token noch rohe Demand-Geometrie preis.

9. Peer-Mesh: Wahl, Web-Relay und Grenzen

Jeder Qt-Client sendet regulär alle 30 Sekunden und bei aktivem Mesh alle 10 Sekunden einen Heartbeat. Ein Heartbeat ist für Wahl und Statusmatrix 90 Sekunden frisch. Der Server verwaltet bis zu 200 Client-Einträge; Einträge laufen nach 600 Sekunden ab. Pro Client akzeptiert PHP maximal einen Heartbeat je zwei Sekunden. Beim geordneten Beenden sendet der Client `action=logout`; bei Stromausfall greift die TTL.

Der Webserver aktiviert das Mesh, wenn der Collector keine frischen Daten liefert oder mindestens die Hälfte frischer Clients `proxy_ok=false` meldet (Proxy-Ausfall oder HTTP 429). Er wählt zufällig einen frischen Kandidaten, dessen direkte Netzwerkverbindung nutzbar ist. Der Heartbeat gibt allen Clients die gemeinsame Rolle und Relay-ID über `role`, `ersatz`, `mesh_active` und `hb` zurück.

Der gewählte Client ruft Verkehr über die normale Fallback-Kette ab und sendet ihn etwa alle fünf Sekunden an `action=relay_push`. Nur die aktuell gewählte UID wird akzeptiert. Die anderen Clients rufen `action=relay_pull` ab; PHP filtert die Antworten nach BBox bzw. Radius. Relay-Daten gelten nach 25 Sekunden als veraltet.

Fällt das Relay länger als 45 Sekunden beim Heartbeat aus oder meldet `net_ok=false`, kann der Server den nächsten Kandidaten wählen. Ein Client, der wegen direkter Quellen-Rate-Limits zurücktritt, ist für 120 Sekunden nicht erneut wählbar. Sobald der Collector wieder frische Daten liefert, kehren Clients in den Normalbetrieb zurück. Das Mesh ist vollständig von der Erreichbarkeit des PHP-Webhosts abhängig; bei Host-Ausfall bleiben nur die direkten Client-Fallbacks adsb.lol → adsb.fi → OpenSky und zuletzt der lokale Cache.

`peer_net.enabled` und `peer_net.name` stehen in `colors.json` oder QSettings. `LTU_MESH=off` schaltet das Mesh ab. Es gibt keine Client-zu-Client-Sockets, UDP-Verbindungen oder offenen Routerports.

10. Repository und Modulübersicht

Datei/Ordner	Verantwortung
`main.py`	PySide6-Hauptfenster, Tabellen-/Detailansicht, Karten-WebView, Threadkoordination und Qt-Bridge.
`settings_dialog.py`	Mehrseitiger Benutzereinstellungsdialog.
`traffic_core.py`	UI-unabhängige Datenabfragen, Normalisierung, Filter, Suche, Tracks, Routen und Snapshots.
`msfs_ai.py`	SimConnect-Verbindung, Spielerposition, AI-Spawns, Updates, Despawn und Parkflugzeuge.
`aircraft_config.py`	Simulatorpfad-Erkennung und `aircraft.cfg`-Titelindex.
`model_map.py`, `modelmap.txt`	VMR/FSLTL-Titelregeln und Auswahl eines lokalen Modells.
`routes.py`	adsbdb-Routenabfrage, Cache und Flugplatznähe-Hilfsfunktionen.
`peer_net.py`	Heartbeats, Clientrolle, Web-Relay Push/Pull und Mesh-Failover.
`app_logging.py`	Sitzungsprotokoll, Aktionen und Fehler-/Recovery-Status.
`startup_splash.py`	Animierter Startbildschirm und tatsächliche Ladeschritte.
`gui_web/`	`map.html`, JavaScript-Karte, SVG-Flugzeugsymbole und vendorte Leaflet-Dateien.
`proxy/index.php`	PHP-API, Statusseite, Admin-Konsole, Dateien, Demand und Mesh-Wahl.
`proxy/client_demand.php`, `proxy/aircraft_tracks.php`	Private Demand-Speicherung und einstündige Flugspuren.
`proxy/ltu_collector.cpp`	Windows-Collector mit WinHTTP, Providerabfragen und Token-Push.
`scripts/`	Token-Generierung/-Anwendung, Kartenpaketprüfung und CI-Verteilung.
`tests/`	Python-, PHP-, C++- und Qt-Mock-Tests.
`.github/workflows/`	CI, vollständiger rea-Windows-Build und Desktop-Release.

10.1 Qt- und JavaScript-Kommunikation

Python sendet JavaScript Karten-/Einstellungszustände über Qt-Signale `stateUpdated` und `settingsUpdated`. JavaScript ruft Qt-Slots des `bridge`-Objekts auf: `aircraftClicked`, `mapContextMenu`, `mapMoved` und `uiReady`. Daten werden als JSON-fähige Zustandsobjekte aus dem UI-unabhängigen Kern bereitgestellt. Dadurch lässt sich `traffic_core.py` ohne Qt testen; `tests/mock_qt_test.py` prüft die Qt-Verdrahtung mit gestubbten PySide6-Objekten.

11. Entwicklung, Tests und Release

11.1 Python-Abhängigkeiten

`requirements.txt` enthält `SimConnect`, `requests` und `PySide6`. Installation aus dem Repository:

```
git clone https://github.com/PhW3130/MSFS-Live-Traffic-Utility-PUBLIC-BETA.git
cd MSFS-Live-Traffic-Utility-PUBLIC-BETA
python -m pip install -r requirements.txt
python main.py
```

Kartenmodus ist aus dem Quellcode auch auf Linux/macOS vorgesehen; MSFS/SimConnect-Funktionen sind Windows-/Simulator-abhängig.

11.2 Headless-Python-Tests

Die Tests benötigen weder ein Display noch einen aktiven Simulator oder Qt-Laufzeitbibliotheken. `requests` und `pytest` zusätzlich installieren, falls sie nicht bereits verfügbar sind.

```
python -m py_compile \
  main.py settings_dialog.py startup_splash.py aircraft_config.py \
  traffic_core.py routes.py msfs_ai.py model_map.py peer_net.py
python tests/test_traffic_core.py
python tests/test_aircraft_history.py
python tests/test_peer_net.py
python tests/test_msfs_ai.py
python tests/mock_qt_test.py
python -m pytest -q tests/test_aircraft_config.py tests/test_model_map.py
```

Die Tests decken Engine-Logik, Filter, Suche, Tracks, Routen, TCAS-nahe Verarbeitung, Fehler-/429-Pfade, Proxyprotokoll, Mesh-Wahl/Relay, MSFS-Parkfüller, Konfigurations- und Modellzuordnung sowie Qt-Bridge, Einstellungen und CSV ab.

11.3 C++- und PHP-Tests

C++-Hilfsbibliotheken mit einem C++11-Compiler testen:

```
for test in opensky_auth client_demand adsb_sources collector_reliability; do
  g++ -std=c++11 -Wall -Wextra -pedantic \
    "tests/test_${test}.cpp" -o "/tmp/test_${test}" && "/tmp/test_${test}"
done
```

PHP-Syntax und Endpunkte prüfen:

```
for file in proxy/index.php proxy/client_demand.php proxy/aircraft_tracks.php; do
  php -l "$file"
done
php tests/test_client_demand.php
php tests/test_opensky_proxy.php
php tests/test_aircraft_tracks.php
```

GitHub Actions führt Python-, PHP-, C++- und Qt-Wiring-Checks bei Pushes und Pull Requests aus. Der Windows-`rea`-Workflow baut und validiert zusätzlich Collector und Client.

11.4 Windows-Pakete und Releases

`build_exe.bat` baut mit PyInstaller einen eigenständigen Windows-Ordner unter `dist/MSFS Live Traffic Utility/`; Python muss auf dem Ziel-PC nicht installiert sein. Das Paket enthält die Karte, Theme, Modellzuordnung, SimConnect-DLL und sichtbare Lizenz-/Drittanbieterhinweise. Versionstags mit Muster `v*` erzeugen ein Windows-Release mit `MSFS-Live-Traffic-Utility-win64.zip`.

`build_all.bat` erzeugt zusätzlich den vollständigen Self-Hosting-Ordner `rea/`. Der GitHub-Actions-Workflow für `rea` validiert Pull Requests. Auf `main` wird ein sicherer Build mit nicht geheimem Platzhaltertoken erstellt; generierte Dateien werden in größenbegrenzten Commits über einen temporären Branch übertragen und erst nach vollständigem Upload zurückgeführt. Das Paket ist ohne privates Token nicht produktiv einsetzbar. Es wird kein Actions-Artefakt für den kompletten `rea`-Build hochgeladen.

11.5 Mitwirken

Fehlerberichte, Modellzuordnungen, Release-Tests und Verbesserungsvorschläge können als Issue oder Pull Request im GitHub-Repository eingereicht werden. Vor einer Veröffentlichung die Veröffentlichungs-Checkliste prüfen und Lizenz, Herkunft sowie Weitergaberechte aller gebündelten Drittanbieter-Inhalte dokumentieren.

12. Sicherheit, Datenschutz und Betrieb

- Echte OpenSky-Zugangsdaten und `token.txt` nie in Git, Clientordner, Release-ZIPs oder öffentlich erreichbare Credential-Dateien legen.
- Token-geschützte PHP-Anfragen nur über vertrauenswürdiges Netz oder TLS/HTTPS übertragen.
- PHP-Dateisystemzustände und GZip-Snapshots vor direktem HTTP-Download schützen; PHP selbst benötigt Schreibrechte.
- Private Demand-Datei außerhalb des Webroots halten. Öffentliche Status- und Demandantworten dürfen weder rohe Clientkoordinaten noch Client-IP-Adressen offenlegen.
- Keine eingehenden Client- oder Collector-Verbindungen auf dem Router freigeben; die Komponenten nutzen ausgehendes HTTP zum Webhost.
- API-Quoten und Geschäfts-/Nutzungsbedingungen nicht umgehen. Providerregeln und Credits können sich ändern; aktuelle Anbieterbedingungen haben Vorrang.
- Beim Tokenwechsel PHP und Collector-INI gemeinsam neu bauen und deployen.

13. Fehlerbehebung

Symptom	Prüfen / Maßnahme
Collector-Aktionen liefern HTTP 401	`LTU_TOKEN` in der PHP-Datei und `token` in `ltu_collector.ini` müssen identisch sein. Nach Rotation beide Komponenten neu bauen und deployen.
Keine neuen Proxy-Daten	Collector pausiert ohne frische `action=traffic`-Demand. Client-URL, Demand-Poll, PHP-Schreibrechte und Ablaufzeit von 20 Sekunden prüfen.
PHP meldet Schreibfehler	Schreibrechte für Relay-Ordner und System-Temp-Verzeichnis prüfen; Webhost darf interne JSON-/GZip-Dateien nicht öffentlich ausliefern.
OpenSky nicht verfügbar oder HTTP 429	JSON-Credentials auf dem Collector-PC, OAuth-Erneuerung, `opensky_interval` und `opensky_quota.txt` prüfen. Retry-After-Wartezeit abwarten; Ledger nicht löschen.
Peer-Mesh startet nicht	PHP-Webhost muss erreichbar sein; `peer_net.enabled` und `LTU_MESH` prüfen. Ohne Webhost kann der Server keine gemeinsame Rolle wählen.
Relay nicht frisch	Heartbeat/Netzstatus und Relay-Logs prüfen. Relay-Daten verfallen nach 25 Sekunden; der Server kann nach 45 Sekunden ohne frischen Relay-Heartbeat neu wählen.
SimConnect verbindet nicht	MSFS läuft? Kompatible DLL verfügbar? `MSFS_SIMCONNECT_DLL` korrekt? Eine geänderte DLL-/Community-Ordner-Angabe benötigt Neustart.
Kein AI-Modell gefunden	Community-Pfad, `aircraft.cfg`-Titel, `modelmap.txt` und lokal installierte FSLTL/VMR-Modelle prüfen. Das Projekt enthält keine Flugzeugmodelle.
Kein eigener Proxy im Client	`LTU_PROXY_URL`, gespeicherte URL, `page.txt` und `colors.json`-Auflösung prüfen. In fertigen Paketen muss die mitgelieferte `page.txt` bzw. Build-Konfiguration auf den gewünschten Host zeigen.
Keine Parkfüller	Spieler am Boden, Flugplatz-Erkennung, SimConnect-Status, Modell-Titel und `MSFS_MAX_PARKED_FILLERS` prüfen. Abgelehnte Spawn-Anfragen können durch Simulatorlimits entstehen.

Collector-Ausgabe erscheint beim direkten Doppelklick in der Konsole. Der Watchdog schreibt zusätzlich `logs/watchdog.log` sowie zeitgestempelte stdout-/stderr-Dateien.

14. Roadmap

- PlaneTrack.ai als mögliche optionale vierte Datenquelle in einer zukünftigen Version.
- Modulare Livery-Auflösung für MSFS-2024-SimObjects, statt ausschließlich ältere `aircraft.cfg`-Titel zu verwenden.
- Die Roadmap ist keine Garantie für Termin oder Implementierung; die aktuell dokumentierte Quellen-Kette bleibt adsb.lol → adsb.fi → OpenSky.

15. Drittanbieter, Lizenz und Haftungsausschluss

Projektcode von PhW3130 steht unter **GNU Affero General Public License v3.0 only (AGPL-3.0-only)**. Der vollständige Lizenztext liegt in `../LICENSE`. Projektcode darf unter den Bedingungen der Lizenz genutzt, geändert und weitergegeben werden. Wenn Nutzer mit einer modifizierten Version über ein Netzwerk interagieren, verlangt AGPLv3 Abschnitt 13, diesen Nutzern den entsprechenden Quellcode anzubieten. Frühere Kopien behalten ihre jeweilige Lizenz.

Leaflet 1.9.4 steht unter BSD-2-Clause; Lizenzdatei: `../gui_web/leaflet/LICENSE`. Der PDF-Build bettet bei vorhandenen Systemschriften Teilmengen von DejaVu Sans/DejaVu Sans Mono ein; DejaVu-Änderungen sind gemeinfrei, die Bitstream-Vera-Lizenzbedingungen stehen in `DejaVu-Font-LICENSE.txt`. Drittanbieterkomponenten, Daten, Kacheln und Dienste werden durch die Projektlizenz nicht neu lizenziert. Details und Hinweise enthält `../THIRD_PARTY_NOTICES.txt` sowie die Veröffentlichungs-Checkliste.

Nutzungsbedingungen und Attribution:

- adsb.fi Open Data ist für persönliche, nicht-kommerzielle Nutzung mit Quellenangabe beschrieben.
- OpenSky Live-API ist laut Anbieter für Forschung und nicht-kommerzielle Nutzung bestimmt.
- adsb.lol unterliegt dynamischen Last-/Nutzungsgrenzen; Antworten und künftige Schlüsselanforderungen können sich ändern.
- Kartenkachel-Anbieter haben separate Nutzungsbedingungen und Attribution. OpenStreetMap-, CARTO- und OpenTopoMap-Hinweise stehen in der README und auf der Karte.
- FSLTL-Flugzeugmodelle sind nicht Bestandteil dieses Projekts.

Haftungsausschluss: MSFS Live Traffic Utility ist nicht mit Microsoft, Asobo Studio, Flightsim.to, FSLTL, ADS-B Exchange, adsb.lol oder anderen Drittorganisationen verbunden und wird von diesen nicht unterstützt. Microsoft Flight Simulator und zugehörige Marken gehören ihren jeweiligen Inhabern. Die Anwendung dient nur der Flugsimulation und Information, nicht Navigation, Disposition oder Flugsicherung. Sie wird ohne Gewährleistung und vorbehaltlich der jeweiligen Lizenz- und Drittanbieterbedingungen bereitgestellt.

16. Begriffe

Begriff	Bedeutung
ADS-B	Automatic Dependent Surveillance–Broadcast; ausgesendete Flugzeugpositions- und Identifikationsdaten.
BBox	Begrenzungsrechteck eines Kartenausschnitts; API-Reihenfolge `north,south,east,west`.
Collector	Windows-C++-Prozess, der aktive Regionen abfragt und Antworten zum PHP-Relay pusht.
Demand	Kurzlebige, grob gerasterte Nachfrage nach Verkehr in einem aktuellen Client-Kartenausschnitt.
Mesh	Serverseitig gewähltes Peer-Relay über HTTP/PHP; keine direkten Peer-Sockets.
OpenSky OAuth2	Collector-Zugang per `client_credentials`, getrennt vom privaten Collector-PHP-Token.
SimConnect	Schnittstelle zwischen Desktopprogramm und Microsoft Flight Simulator.
TCAS	Verkehrsanzeige relativ zur eigenen Spielerposition; kein echtes Luftfahrtsicherungsgerät.
`rea/`	Von `build_all.bat` erzeugte Verteilung für PHP-Host, Collector-PC und Client-PCs.

Anhang A. Lizenzhinweis für eingebettete PDF-Schriften

Der PDF-Build bettet bei verfügbaren Systemschriften Teilmengen von DejaVu Sans und DejaVu Sans Mono ein. Der folgende vollständige Lizenzhinweis wird deshalb mit diesem Dokument weitergegeben.

The generated PDF embeds subsets of DejaVu Sans and DejaVu Sans Mono when those fonts are installed on the build system. DejaVu changes are in the public domain. The fonts are based on Bitstream Vera; Bitstream Vera is a trademark of Bitstream, Inc.

Copyright (c) 2003 by Bitstream, Inc. All Rights Reserved.

Permission is hereby granted, free of charge, to any person obtaining a copy of the fonts accompanying this license ("Fonts") and associated documentation files (the "Font Software"), to reproduce and distribute the Font Software, including without limitation the rights to use, copy, merge, publish, distribute, and/or sell copies of the Font Software, and to permit persons to whom the Font Software is furnished to do so, subject to the following conditions:

The above copyright and trademark notices and this permission notice shall be included in all copies of one or more of the Font Software typefaces.

The Font Software may be modified, altered, or added to, and in particular the designs of glyphs or characters in the Fonts may be modified and additional glyphs or characters may be added to the Fonts, only if the fonts are renamed to names not containing either the words "Bitstream" or the word "Vera".

This License becomes null and void to the extent applicable to Fonts or Font Software that has been modified and distributed under the "Bitstream Vera" names.

The Font Software may be sold as part of a larger software package but no copy of one or more of the Font Software typefaces may be sold by itself.

THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK OR OTHER RIGHT. IN NO EVENT SHALL BITSTREAM OR THE GNOME FOUNDATION BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

Except as contained in this notice, the names of Gnome, the Gnome Foundation, and Bitstream Inc. shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Font Software without prior written authorization from the Gnome Foundation or Bitstream Inc., respectively. For further information, contact: fonts at gnome dot org.